CS 10: Operating Systems Laboratory – Assessment Task 8

Process States and Memory Management

Name: Rodnee Glen A. Martin

Year & Section: 3 BSCS A

Simulator: YASMIN CPU-OS Simulator 7.5.50

Date Performed: October 23, 2026

All exercises were done in the simulator. The programs ForeverLooping, OSQueuesDemo and WaitOnReadDemo were compiled from the Appendix sources with the built-in compiler and saved as .sas files. Figures are screenshots of the simulator; values in the tables are read from the simulator windows.

LO 1: Process States and Transitions

Settings: Process States window with **Stay on top** and **Animate** checked; one process of ForeverLooping (Pid 3) created; **Suspend on state change** checked in the READY PROCESSES area only; CPU speed at the fastest position.

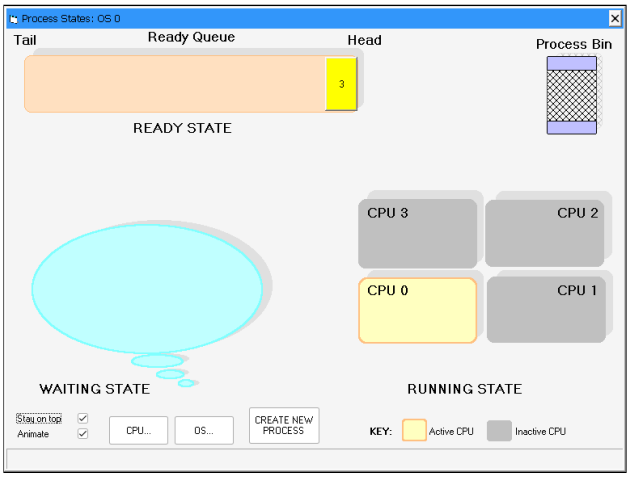


Figure 1. Starting point: process 3 at the head of the Ready Queue (READY state).

#	Action	Resultant State	Success (Y/N)	Fail (Y/N)
1	Drag process to the waiting queue	READY	N	Y
2	Drag process to the Process Bin	READY	N	Y
3	Drag process to the CPU 0 box	RUNNING	Y	N
4	Drag process to the waiting queue	WAITING	Y	N
5	Drag process to CPU 0	WAITING	N	Y
6	Drag process to the Process Bin	WAITING	N	Y
7	Drag process to the Ready Queue	READY (scheduler suspended on the change)	Y	N
8	Click RESUME	READY (OS resumed)	Y	N
9	Drag process to the ready queue	READY (no change)	N	Y
10	Click RESUME	READY	Y	N
11	Drag process to the Process Bin	READY	N	Y

Actions 1, 2, 5, 6 and 11 showed the message below in the status bar. Action 9 showed no message; the process simply stayed at the head of the ready queue because READY to READY is not a state change.



Figure 2. Status bar after an illegal drag (action 1).



Figure 3. Action 3: process 3 RUNNING on CPU 0 (left). Action 4: process 3 in the WAITING state (right).

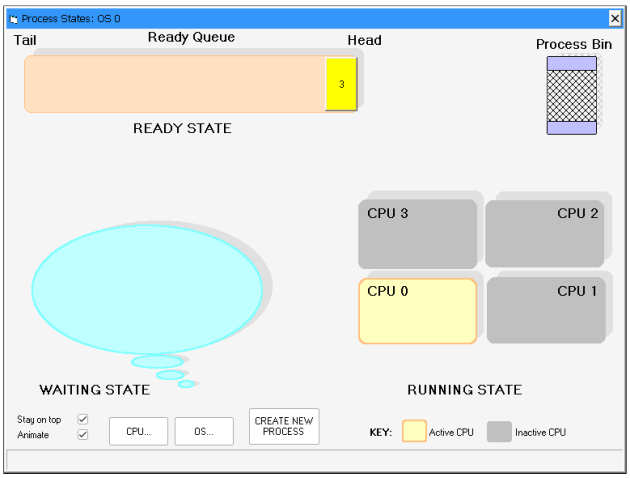


Figure 4. Action 7: process 3 back in the Ready Queue (READY). The simulator showed the message "OS 0 scheduler is suspended on process 3 changing state to READY".

List the legal state transitions:

From State	To State
READY	RUNNING
RUNNING	WAITING
WAITING	READY
RUNNING	READY (pre-empted, e.g. time slice used up)
RUNNING	Terminated (Process Bin)

The first three were tested successfully by dragging (actions 3, 4 and 7). RUNNING to READY is done by the scheduler when a process is pre-empted, and a process only leaves the system (Process Bin) after it has run. Illegal: READY to WAITING, READY to Bin, WAITING to RUNNING and WAITING to Bin.

State diagram (illegal transitions removed):

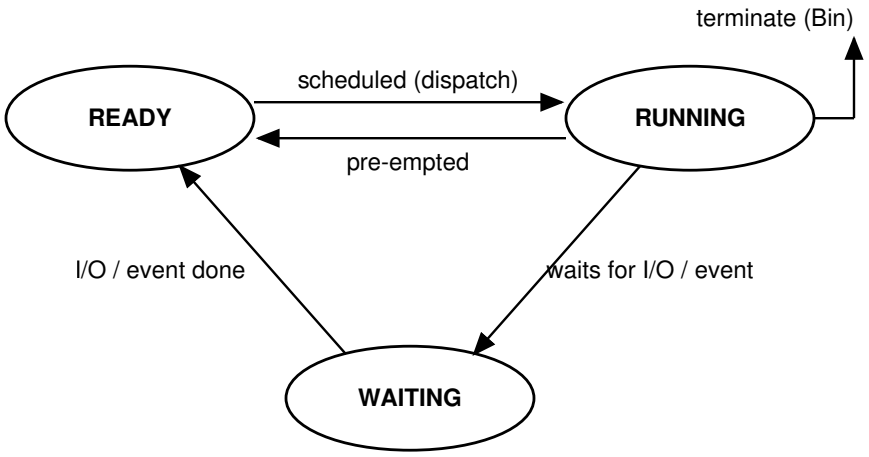


Figure 5. Legal process state transitions.

LO 2: Process States in Action

OSQueuesDemo

One OSQueuesDemo process was created and all three **Suspend on state change** boxes were checked. After START, the OS stopped at every state change; RESUME was clicked after each one.

#	State	Simulator message
1	RUNNING	suspended on process 3 changing state to RUNNING
2	WAITING	suspended on process 3 changing state to WAITING
3	READY	suspended on process 3 changing state to READY
4	RUNNING	suspended on process 3 changing state to RUNNING
5	WAITING	suspended on process 3 changing state to WAITING
6	READY	suspended on process 3 changing state to READY



Figure 6. The first three state-change messages: RUNNING, WAITING, READY.

Explain the sequence of states:

The process runs the for loop 15 times (RUNNING). Then wait(3) makes it wait 3 seconds, so the OS takes it off the CPU and puts it in the WAITING queue. When the 3 seconds are over, the wait is finished and the process goes to the READY queue. The scheduler then gives it the CPU again (RUNNING). Because of the forever loop, the cycle RUNNING → WAITING → READY repeats.

WaitOnReadDemo

No suspend boxes checked, fastest speed. The Console (INPUT OUTPUT… in the CPU Simulator) and the Keyboard (SHOW KEYBD…) were opened with Stay on top, then START.

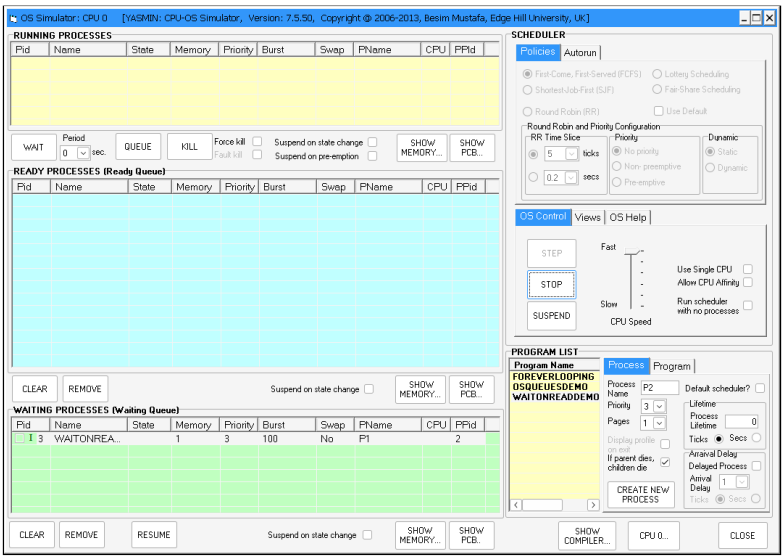


Figure 7. At the ">" prompt WaitOnReadDemo is in the WAITING queue.

State of the process while the prompt ">" is shown:

WAITING. The process is blocked in read(d) and waits for a key from the keyboard.

State of the process immediately after a key is pressed:

READY. As soon as the key was pressed the process left the WAITING queue and went into the READY queue (shown by checking Suspend on state change in the READY area: "OS 0 scheduler is suspended on process 3 changing state to READY"). The scheduler then moved it to RUNNING; it printed the key code and the next ">" prompt, and went back to WAITING.

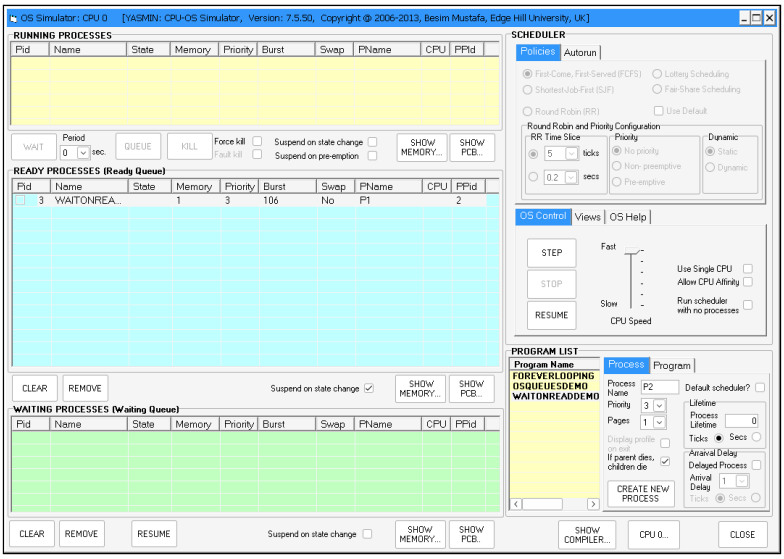


Figure 8. After pressing a key, the process moved from WAITING to READY.

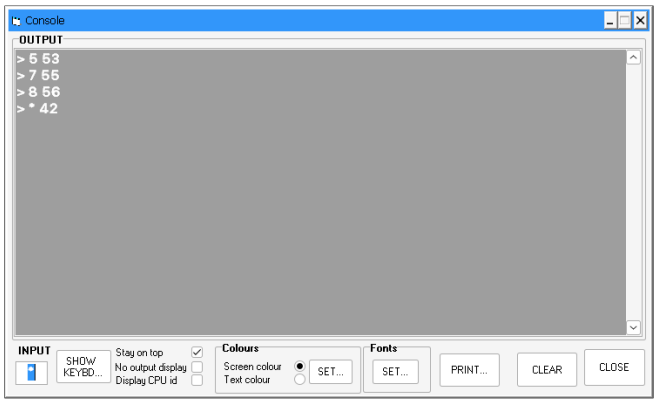


Figure 9. Console output. read(d) stores the character code, so 5 → 53, 7 → 55, 8 → 56; "" is code 42, which ends the program.*

Explanation:

A process that needs input cannot use the CPU, so the OS puts it in the WAITING state until the I/O event finishes. When the input arrives it becomes READY and runs again only when the scheduler picks it. This keeps the CPU free for other processes while one process waits.

LO 3: Memory Placement Policies

All processes and programs were removed. In VIEW MEMORY…, 10 frames were added (1 frame each), then frames 2, 3, 5, 7, 8 and 9 were removed. This left frames 0, 1, 4 and 6 allocated with three holes: frames 2–3 (512 bytes), frame 5 (256 bytes) and frames 7–9 (768 bytes). ForeverLooping (1 page = 256 bytes) was loaded and one process created and removed for each policy, then RESET was clicked.



Figure 10. Memory with holes: frames 0, 1, 4 and 6 allocated.



Figure 11. Where the process (green) was placed: First Fit → frame 2 (left), Best Fit → frame 5 (middle), Worst Fit → frame 7 (right).

Method	Explanation
First Fit placement method	Searches memory from the start and uses the first hole that is big enough. The process went into frame 2, the first hole (frames 2–3, 512 bytes), even though a smaller hole existed. It is the fastest method because the search stops early.
Best Fit placement method	Searches all holes and uses the smallest hole that is big enough. The process went into frame 5, the 256-byte hole that fits it exactly, leaving the bigger holes for bigger processes. It wastes the least space but leaves tiny leftover holes.
Worst Fit placement method	Searches all holes and uses the largest hole. The process went into frame 7, the start of the biggest hole (frames 7–9, 768 bytes). The leftover part of the hole stays big enough to be useful for another process.

LO 4: Virtual Memory and Swapping

VIEW UTILIZATION… and VIEW MEMORY… were opened and **Paging Enabled** was unchecked. RAM is 2560 bytes (10 frames of 256 bytes). Three OSQueuesDemo processes were created:

Process	Pages	Free	Alloc	Swap
P1	4	1536	1024	0
P2	5	256	2304	0
P3	3	256	2304	768

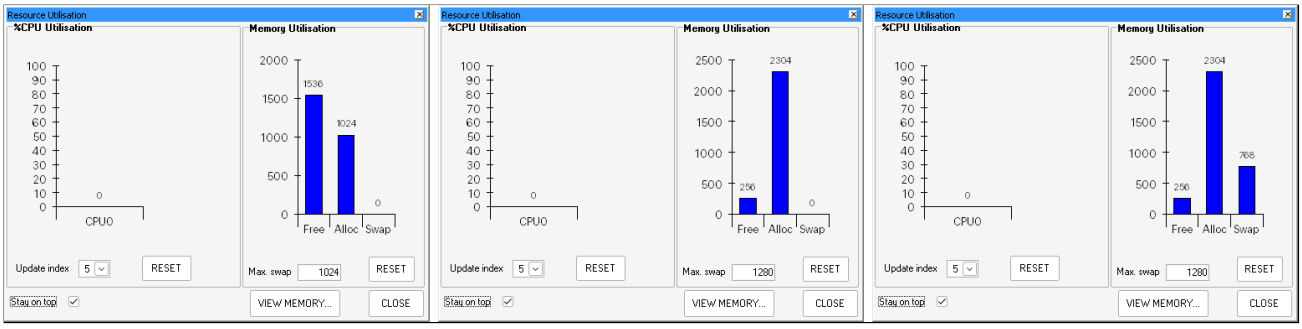


Figure 12. Resource Utilisation after creating P1, P2 and P3.

Comment on why P3's memory is swapped out:

P1 (4 pages = 1024 bytes) and P2 (5 pages = 1280 bytes) used 2304 of the 2560 bytes of RAM, so only 256 bytes (1 frame) were free. P3 needs 3 pages = 768 bytes, which does not fit in 256 bytes. With paging off, a process must be loaded as one piece, so the OS put all of P3's 768 bytes in the swap area on disk (virtual memory). Free and Alloc did not change; Swap went from 0 to 768, and the Swap column of P3 shows "Yes".



Figure 13. RAM after the three processes: P1 in frames 0–3, P2 in frames 4–8, frame 9 free; Swapped = 768.

Then **Suspend on state change** was checked in the RUNNING area only, speed at fastest, and START was clicked. Values were noted at each message and RESUME was clicked:

Process	Free	Alloc	Swap
P1	256	2304	768
P2	256	2304	768
P3	768	1792	1280
P1	768	1792	1280
P2	256	2304	768
P3	768	1792	1280

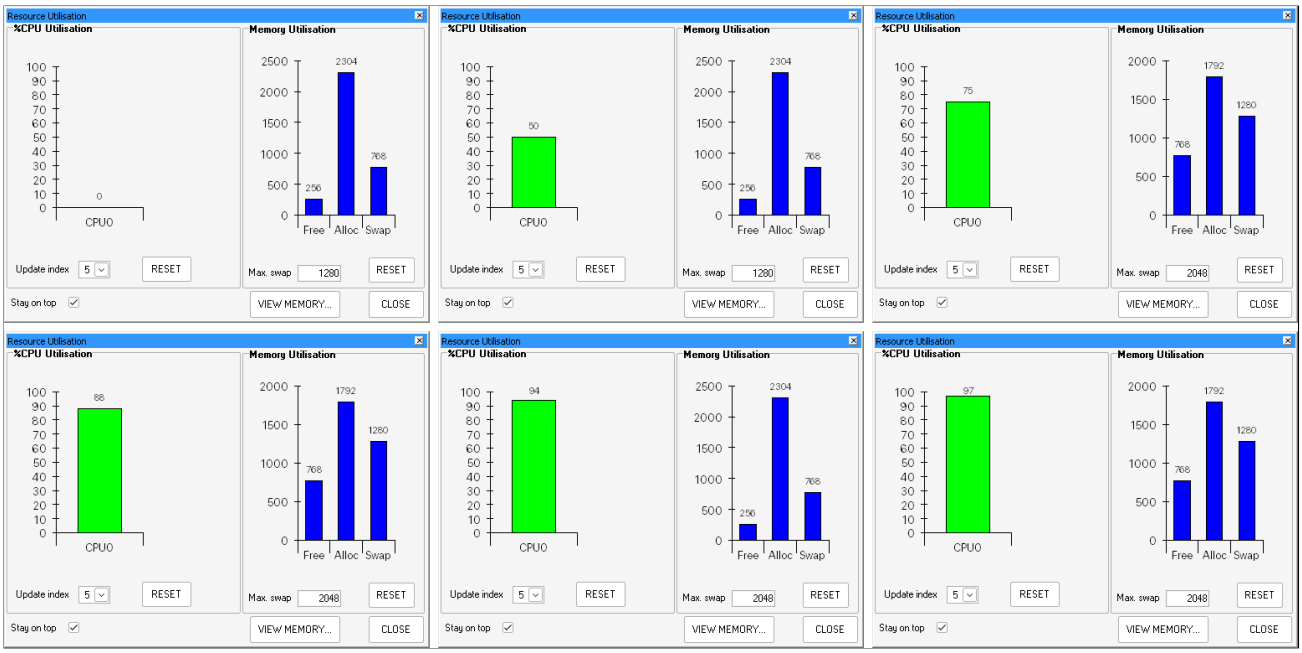


Figure 14. Resource Utilisation at the six RUNNING messages (P1, P2, P3 on the top row; P1, P2, P3 on the bottom row).

Then the box was unchecked, RESUME clicked and every process was killed with KILL as it ran:

CPU %	Free	Alloc	Swap
0	2560	0	0

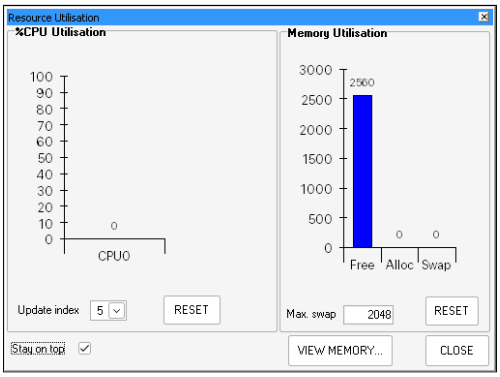


Figure 15. After all three processes were killed.

Comments on how virtual memory management works:

When RAM is full, the OS does not refuse new processes. It uses part of the disk as an extension of RAM (virtual memory) and swaps a process's memory out to it, as happened to P3. A swapped-out process cannot run from disk, so when the scheduler picks it, its memory must first be swapped back into RAM. If there is no room, another process that is not running is swapped out to make space. In the run, when P3 was scheduled, P2 (1280 bytes) was swapped out and P3 (768 bytes) was swapped in: Alloc fell to 1792, Free rose to 768 and Swap rose to 1280. When P2 ran again, P3 was swapped out and P2 swapped back in (back to 2304 / 256 / 768). P1 always fit and stayed in RAM. Swapping lets more processes exist than RAM can hold, but each swap costs disk time, so too much swapping slows the system. When the processes were killed, all their memory was freed: Free went back to 2560, and Alloc, Swap and CPU % went to 0.

Appendix – Program Sources

```
program OSQueuesDemo
  while true
    for n = 1 to 15
      i = 1
    next
    wait(3)
  wend
end
```

```
program WaitOnReadDemo
  regvar d integer
  while true
    for n = 1 to 15
      i = 1
    next
    write("> ")
    read(d)
    writeln(d)
    if d = 42 then
      break
    end if
  wend
end
```

```
program ForeverLooping
  while true
    n = 1
  wend
end
```